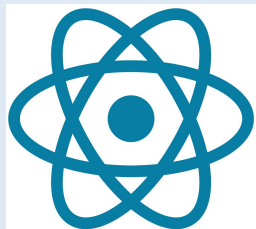


# Mòdul 06



**Institut TIC**  
de Barcelona

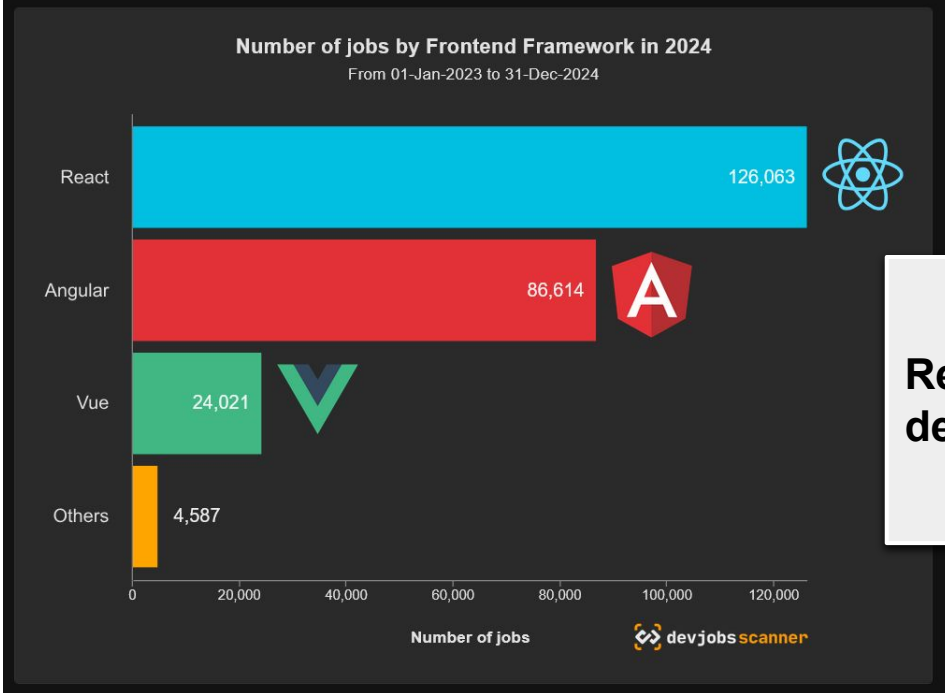


Desenvolupament web en entorn  
client

**RA 7.2 - FRAMEWORKS / Introducció ReactJS**

1. Introducció
2. ReactJs
3. Instal·lar ReactJs
4. Jsx
5. Components i props
6. Hooks
7. Formularis
8. Routing

## Which is the Most Demanded Frontend Framework?



**React és la biblioteca amb més demanda durant el 2024**

- És una **llibreria** Open Source de **JavaScript** creada per **Facebook**.
- Molt popular per **construir interfícies d'usuari (UI)** dinàmiques a través de **components**.
- **Rapidesa i facilitat** en crear prototips i components.
- Està constantment en manteniment per la **comunitat**.
- Actualment està en la versió **19**..
- Es basa en **SPA** (Single Page Application).
  - Es carrega una sola pàgina HTML i React actualitza el contingut dinàmicament sense recarregar la pàgina.

- **Components:** Són blocs d'html/css reutilitzables en el projecte.
- **JSX:** Sintaxi que permet escriure HTML dins de JavaScript. Això facilita la creació de components i la visualització del codi.
- **Virtual DOM:** React utilitza un DOM virtual propi (Virtual DOM) per optimitzar el rendiment. En lloc de manipular directament el DOM del navegador, React crea una còpia virtual del DOM i actualitza només les parts que han canviat, millorant l'eficiència.
- **Unidireccionalitat del flux de dades:** Els components van sempre dels components pares als components fills.
- **React Hooks:** Introduïts a la versió 16.8, els Hooks permeten als desenvolupadors utilitzar l'estat i altres característiques de React sense necessitat de escriure classes.

**Actualment no es fan servir classes per crear components!!!**

1. Cal tenir instal·lat [node.js](https://node.js.org/)
2. A l'instal·lar node tenim npm (Node Package Manager)
3. Per temes de seguretat actualment es recomana fer servir **pnpm**:
  - 3.1. `corepack enable`
  - 3.2. `corepack use pnpm@latest-10`

```
C:\Users\isard>node -v  
v22.14.0
```

```
C:\Users\isard>npm -v  
10.9.2
```

Podem crear projectes a través de diferents opcions:

a. ViteJs

i. Crear Projecte `pnpm create vite`

ii. Arrancar el projecte `pnpm run dev`



```
◇ Project name:
  AppDemo
◇ Package name:
  appdemo
◇ Select a framework:
  React
◇ Select a variant:
  JavaScript
◇ Scaffolding project in
  |
  Done. Now run:

  cd AppDemo
  npm install
  npm run dev
```

b. NextJs (Framework de ReactJs)

| Estratègia                           | Com funciona?                                                                                             | Avantatges                                                              | Inconvenients                                                    | Quan usar-la?                                                    |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------------------------------|------------------------------------------------------------------|
| <b>SPA</b> (Single Page Application) | Carrega una sola pàgina HTML; el navegador baixa tot el JS i React actualitza el contingut (Vite + React) | Molt senzilla d'implementar i problemes amb el SEO                      | Càrrega inicial lenta perquè s'ha de baixar tot el JS primer     | Aplicacions internes, dashboards o eines sense necessitat de SEO |
| <b>SSR</b> (Server Side Rendering)   | El servidor genera el HTML complet i l'envia ja renderitzat al navegador (NextJs)                         | Millor rendiment inicial (l'usuari veu el contingut abans) i millor SEO | Més complex de configurar i mantenir                             | Pàgines públiques dinàmiques com notícies o xarxes socials       |
| <b>SSG</b> (Static Site Generation)  | Es generen fitxers HTML estàtics durant la compilació, abans de qualsevol petició (NextJs)                | Molt ràpid i ideal per a SEO                                            | El contingut no és en temps real; cal recompilar per actualitzar | Blogs, documentació o pàgines de màrqueting                      |
| <b>RSC</b> (React Server Components) | Barreja components de servidor (sense JS al client) i interactius de client en un mateix arbre (NextJs)   | Màxim rendiment i menys JavaScript al navegador                         | Tecnologia nova i complexa que requereix experiència             | Aplicacions grans que busquen el rendiment més alt possible      |



```
> node_modules
> public
> src
.eslintrc.cjs
.gitignore
index.html
package-lock.json
package.json
README.md
vite.config.js
```

- Arxius que es creen en la instal·lació del projecte de react:
  - **index.html** ⇒ És l'arxiu html que mostrarà, en format SPA, tot el nostre projecte a través de l'explorador.
  - **public/** ⇒ conté tota la part estàtica utilitzada a l'aplicació (exemple: icones)
  - **src/** ⇒ És on està tota la nostra aplicació.
  - **package.json** ⇒ Arxiu on conté informació del projecte i de les dependències instal·lades a l'aplicació (entre d'altres la versió de React).
  - **.gitignore**

- Com funciona ReactJS

Directori



main.jsx

```
1 import React from 'react';
2 import ReactDOM from 'react-dom/client';
3 import App from './App';
4
5 const root = ReactDOM.createRoot(document.getElementById('root'));
6 root.render(
7   <React.StrictMode>
8     <App />
9   </React.StrictMode>
10 );
```

Mètode **render()** ⇒ Encarregat de descriure com s'ha de veure la UI.

App.js

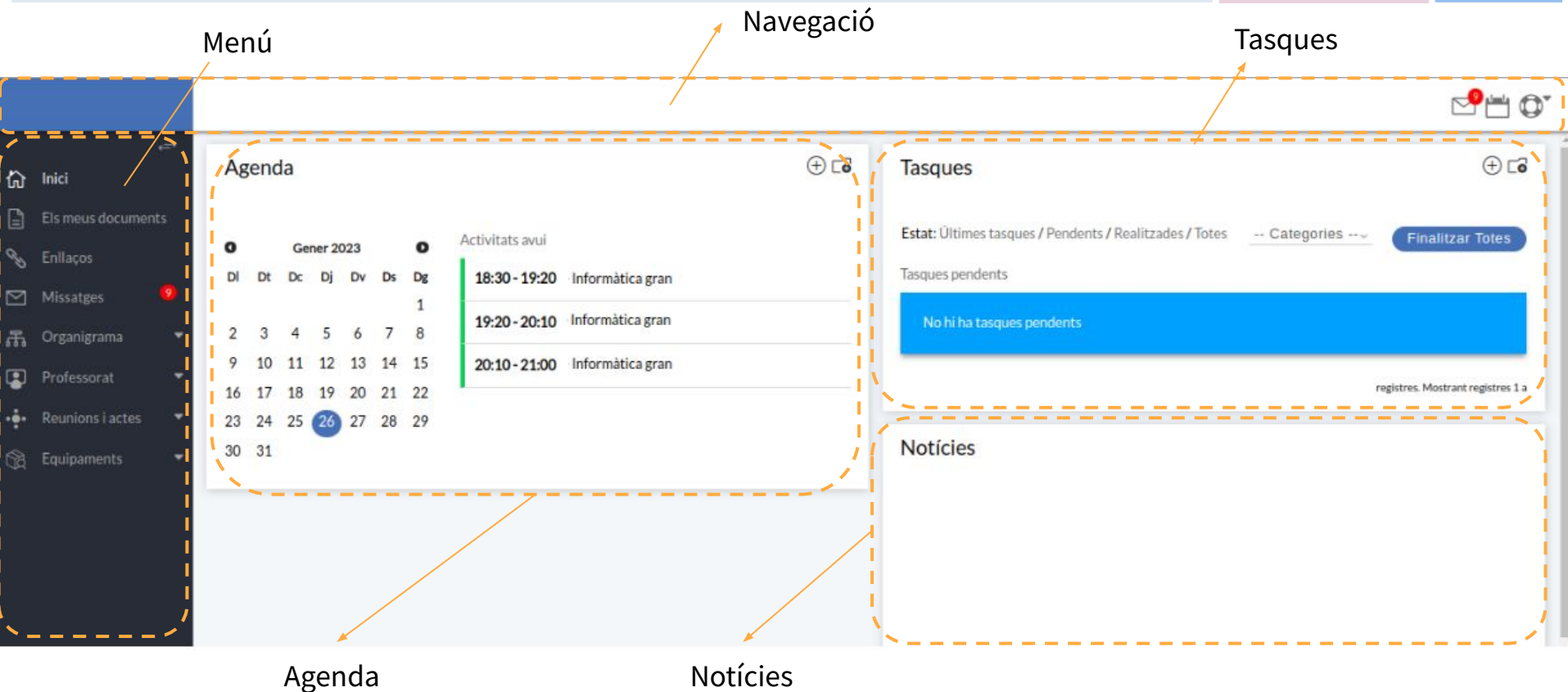
```
1 import './App.css';
2
3 function App() {
4   return <div className="App"> Hello World </div>;
5 }
6
7 export default App;
```

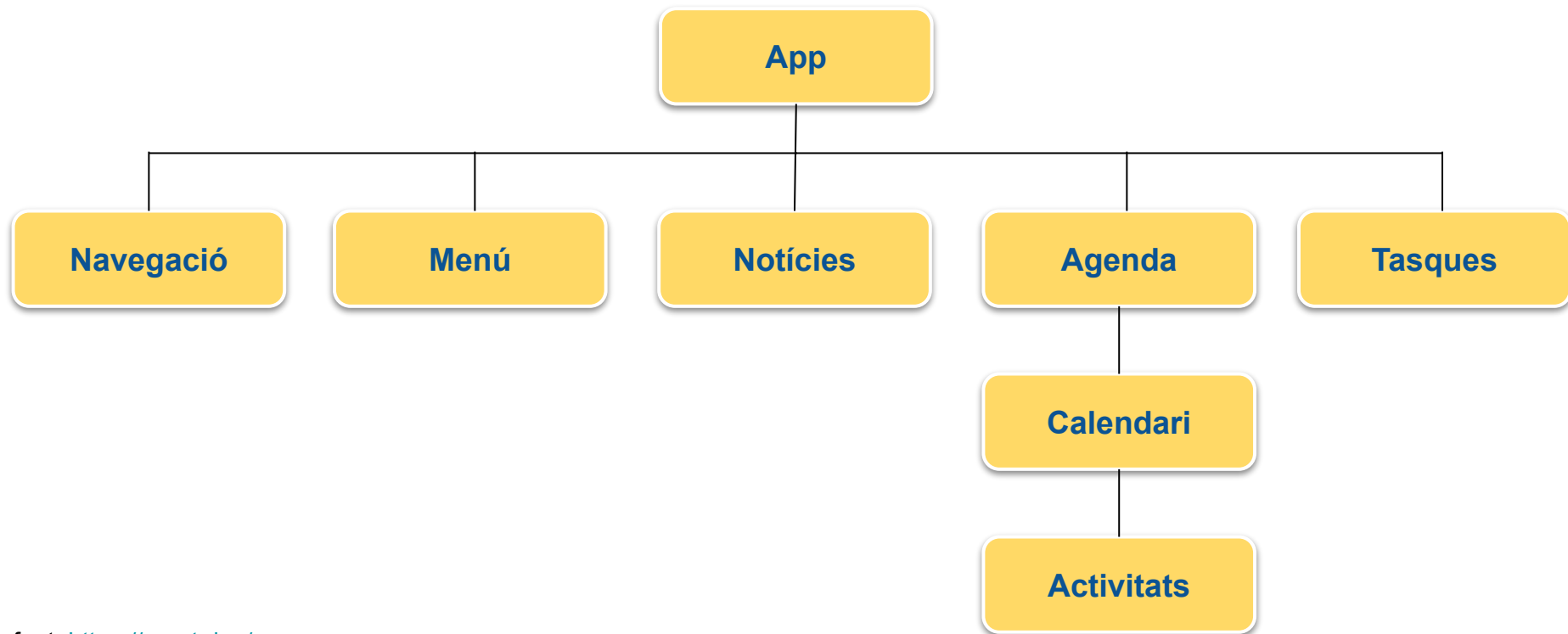
index.html

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

- És una peça modular, aïllat i reutilitzable de codi (html/css/js) que defineix com s'ha de renderitzar una part de la IU.
- Una app de react està creada per un arbre de components.
- **App** ⇒ component **root**. Conté els components anomenats child's.







Els nom del component ha de ser **Pascal-case** (La primera en majúscula).

```
function Usuari(){  
  // CODI JAVASCRIPT  
  const usuari = "marta@example.com"  
  //CODI JSX  
  return (  
    <h1>Hola {usuari}</h1>  
  )  
}  
  
// SEMPRE EXPORTEM LA FUNCIO  
export default Usuari
```

Hem de definir una funció, que serà el component

Podem utilitzar codi javascript (variables, funcions,...)

JSX: Codi que podem barrejar html i javascript.  
Sempre ha de tenir una etiqueta root.

Quan utilitzem javascript dins de JSX hem de fer servir { }

Sempre s'ha d'exportar el component per poder-lo utilitzar

Els components es criden des dels components pares.  
Recordeu que el component principal és App.

```
import Usuari from './Usuari.jsx'
```

S'ha d'importar del lloc on està ubicat en el projecte.  
**Recordeu posar el .jsx!!!**

```
function App(){
```

```
  return (
```

```
    <Usuari />
```

Dins del JSX d'un component pare, podem cridar el component  
com si fos una etiqueta html.

```
  )
```

```
}
```

```
export default App
```

**Hola marta@example.com**

```
function Usuari(){  
  
  // CODI JAVASCRIPT  
  const usuari = "marta@example.com"  
  const getName = function (usuari){  
    return usuari.split("@")[0];  
  }  
  const llistaHabilitats = ["javascript", "html", "css"]  
  
  //CODI JSX  
  return (  
    <div>  
      <h1>Hola {getName(usuari)}</h1>  
      <ul>  
        {  
          llistaHabilitats.map((element, index) => (  
            <li key={index}>{element}</li>  
          ))  
        }  
      </ul>  
    </div>  
  )  
}  
  
// SEMPRE EXPORTEM LA FUNCIO  
export default Usuari
```

Dins del JSX podem utilitzar variables, funcions o iteracions de javascript.

**Per les iteracions:**

- **S'ha de fer servir map!!!**
- **S'ha de definir-li un key per cada element que itera amb l'index.**

**Hola marta**

- javascript
- html
- css



Els props serveixen per passar **informació** del **component pare** al **component fill**.

## Component fill

```
import './Card.css'

function Card(props){

  const name = props.name;
  let preu = props.preu;
  let desc = props.desc;
  let srcImg = "/card/img1.jpg";

  const preuIva = function(preu){
    return (preu * 1.21).toFixed(2);
  }

  return (
    <div className="div-producte">
      <img src= {srcImg} />
      <h1>{name}</h1>
      <h3>Preu: {preuIva(preu)}</h3>
      <p>{desc}</p>
    </div>
  )
}

export default Card;
```

En el component fill rep un objecte props on poden accedir a la informació de cada prop.

En el component pare definim el/s component/s i aquells props que volem utilitzar per enviar informació al component fill.

## Component pare

```
<section className='productes-container'>
  <Card name="Auriculars" preu="1" desc="Portatil per a gamers"/>
  <Card name="Ratoli" preu="88" desc="ratoli amb bluetooth"/>
</section>
```

El **cicle de vida** d'un component és:

1. **Mounting** (Quan el component es renderitza per primera vegada)
2. **Updating** (Quan el component es torna a renderitzar per canvis en props o estat)
3. **Unmounting** (Quan el component es treu de la pantalla)

Són funcions que permeten mantenir **l'estat** i reaccionar als **cicles de vida dels components** funcionals.

- Han de ser cridats sempre a dins del component.
- S'han d'utilitzar sempre en la part superior del component.

- **useState()** : Gestiona l'estat d'un component.
- **useEffect()** : Per efecte secundaris (càrrega d'informació, timers,...)
  - És el hook que s'utilitza per consultes a les Aprest amb fetch
- **useRef()**
- **useContext()**
- **custom hooks...**

